

Building Your First Agent

Building a Simple AI Agent, Part 1

Now that you understand the agent loop and how to craft effective prompts, we can build a simple AI agent. This agent will be able to do five things in a directory, read their content, answer questions about them, list them, and then the agent loop: "list" to retrieve agent, "describe" to actions, "evaluate" them, and "update" to memory—step by step.

The Agent Loop in Python

The agent loop is the backbone of an AI agent, enabling it to perform tasks by combining reasoning, generation, observation, and memory within an iterative process. The action function is the agent loop, and it is the core of the agent's operation and execution.

- Contextual Prompting:** Contextualize the agent's reasoning, user input, and system rules into a single prompt. This ensures the LLM has all the context it needs to decide on the next action, maintaining continuity across iterations.
- Generate Response:** Send the constructed prompt to the LLM and retrieve a response. This response will guide the agent's next step by providing instructions in a structured format.
- Parse Response:** Extract the intended action and its parameters from the LLM's output. The response must adhere to a predefined structure (e.g., JSON format) to ensure it can be interpreted correctly.
- Execute Action:** Use the extracted action and its parameters to perform the requested task with the appropriate tool. This could involve listing files, reading content, or generating a response.
- Generate Result to String:** Convert the result of the executed action into a string. This allows the agent to store the result in its memory and provide clear feedback to the user or itself.
- Conditional Logic:** Implement conditional logic to handle different outcomes from the action and results. The loop may terminate if a "terminate" action is specified or if the agent has completed the task.

The agent iterates through this loop, refining its behavior and adapting its actions until it reaches a stopping condition. This process is what enables the agent to interact dynamically and respond intelligently to users.

Here's how these steps come together in code:

```
# The Agent Loop
def main():
    # 1. Initialize context: combine agent rules with memory
    context = agent_rules + memory

    # 2. Generate response from LLM
    prompt = f"{context}\n\n{user_input}"
    response = generate_response(prompt)

    # 3. Parse response to determine action
    action = parse_response(response)

    # 4. Execute action
    if action["action"] == "list_files":
        result = list_files(directory)
        list_result["list_files"] = result["files"]
    elif action["action"] == "read_file":
        result = read_file(directory, file_path)
        list_result["read_file"] = result["content"]
    elif action["action"] == "generate":
        result = generate_response(prompt)
        list_result["generate"] = result["response"]
    elif action["action"] == "terminate":
        print(f"Terminated")
        return

    # 5. Generate result to string
    result = f"{action['action']} {action['result']}\n\n{list_result}"

    # 6. Update memory with response and results
    memory.append(result)

    # 7. Check termination condition
    if action["action"] == "terminate":
        return

    iterations += 1
```

Step 1: Constructing the Agent Prompt

The prompt is the key to specifying the agent's behavior. It's the current memory of observations that the agent has made in its memory, a description of the task that the agent should perform. This ensures the agent always starts with the context and conditions when deciding what actions to take.

prompt = agent_rules + memory

Explanation:

- agent_rules:** This contains the predefined system instructions, ensuring the agent follows within its defined constraints and understands its tools.
- memory:** This is a record of past observations, including user input, the agent's responses, and the results of executed actions.

By constructing the prompt this way, the agent retains continuity across iterations, ensuring it can adapt its behavior based on previous actions and results. The memory will build up over time, and it's important to manage it to prevent the agent from becoming overwhelmed.

Agent Rules: Defining the Agent's Behavior

Before the agent begins its loop, it must have a clear set of rules that define its behavior, capabilities, and constraints. These **agent rules** are specified in the system message and play a critical role in ensuring the agent interprets prompts correctly and adheres to its defined boundaries.

Here's an example:

The **agent_rules** are defined in a system message that instructs the LLM on how the agent should behave, what tools it has available, and how to format its responses. These rules are included at the start of the prompt for every iteration.

Agent Rules: Defining the Agent's Behavior

Before the agent begins its loop, it must have a clear set of rules that define its behavior, capabilities, and constraints. These **agent rules** are specified in the system message and play a critical role in ensuring the agent interprets prompts correctly and adheres to its defined boundaries.

```
agent_rules = """
You are an AI agent that can perform tasks by using available tools.

Available tools:
- list_files(directory): List all files and the current directory.
- read_file(directory, file_path): Read the content of a file.
- generate_response(prompt): Generate a response to a prompt.

If a user asks about files, first use list_files to get the agent's context and then use read_file to read the file.

Every response MUST have an action.
Respond in this format:

{
  "action": "list_files",
  "result": "list_files_result"
}
```

Explanation:

- Role of system message:** The **system** role is the message for a user to establish ground rules for the agent. This message tells the LLM what it can do and how it should behave throughout the session.
- Tools description:** The **tools** role is used to tell the LLM what tools the agent can use, providing a structured list of tools and their descriptions.
- Output format:** The **output** role is used to tell the LLM what format the agent should use for its responses, ensuring consistency and readability.

Each of the 'tools' in the system prompt corresponds to a function in the code. The agent's goal is to use these tools to interact with the user and the environment, following the instructions that are provided in the system prompt.

The agent's understanding of the tools and their parameters is what enables it to make decisions about which tool to use and what parameters to pass to it. This is the core of the agent's reasoning process.

Before the agent begins its loop, it must have a clear set of rules that define its behavior, capabilities, and constraints. These **agent rules** are specified in the system message and play a critical role in ensuring the agent interprets prompts correctly and adheres to its defined boundaries.

For example, if the user asks the agent to read the contents of a specific file, the agent will first use the **list_files** tool to get the list of files in the current directory. Then, based on the result, it will use the **read_file** tool to read the content of the specific file.

The agent's goal is to use these tools to interact with the user and the environment, following the instructions that are provided in the system prompt.

This separation of reasoning and action is what makes the agent loop so powerful—it allows the agent to make decisions about which tool to use and what parameters to pass to it.

Additionally, the agent's memory allows it to store the results of its actions, which it can use to make decisions about which tool to use and what parameters to pass to it.

The agent's memory is what allows it to store the results of its actions, which it can use to make decisions about which tool to use and what parameters to pass to it.

The agent's memory is what allows it to store the results of its actions, which it can use to make decisions about which tool to use and what parameters to pass to it.

The agent's memory is what allows it to store the results of its actions, which it can use to make decisions about which tool to use and what parameters to pass to it.

The agent's memory is what allows it to store the results of its actions, which it can use to make decisions about which tool to use and what parameters to pass to it.

The agent's memory is what allows it to store the results of its actions, which it can use to make decisions about which tool to use and what parameters to pass to it.

The agent's memory is what allows it to store the results of its actions, which it can use to make decisions about which tool to use and what parameters to pass to it.

The agent's memory is what allows it to store the results of its actions, which it can use to make decisions about which tool to use and what parameters to pass to it.

The agent's memory is what allows it to store the results of its actions, which it can use to make decisions about which tool to use and what parameters to pass to it.

The agent's memory is what allows it to store the results of its actions, which it can use to make decisions about which tool to use and what parameters to pass to it.

The agent's memory is what allows it to store the results of its actions, which it can use to make decisions about which tool to use and what parameters to pass to it.

The agent's memory is what allows it to store the results of its actions, which it can use to make decisions about which tool to use and what parameters to pass to it.

The agent's memory is what allows it to store the results of its actions, which it can use to make decisions about which tool to use and what parameters to pass to it.

The agent's memory is what allows it to store the results of its actions, which it can use to make decisions about which tool to use and what parameters to pass to it.

The agent's memory is what allows it to store the results of its actions, which it can use to make decisions about which tool to use and what parameters to pass to it.

The agent's memory is what allows it to store the results of its actions, which it can use to make decisions about which tool to use and what parameters to pass to it.

The agent's memory is what allows it to store the results of its actions, which it can use to make decisions about which tool to use and what parameters to pass to it.

The agent's memory is what allows it to store the results of its actions, which it can use to make decisions about which tool to use and what parameters to pass to it.

The agent's memory is what allows it to store the results of its actions, which it can use to make decisions about which tool to use and what parameters to pass to it.

The agent's memory is what allows it to store the results of its actions, which it can use to make decisions about which tool to use and what parameters to pass to it.

The agent's memory is what allows it to store the results of its actions, which it can use to make decisions about which tool to use and what parameters to pass to it.

The agent's memory is what allows it to store the results of its actions, which it can use to make decisions about which tool to use and what parameters to pass to it.

The agent's memory is what allows it to store the results of its actions, which it can use to make decisions about which tool to use and what parameters to pass to it.

The agent's memory is what allows it to store the results of its actions, which it can use to make decisions about which tool to use and what parameters to pass to it.

The agent's memory is what allows it to store the results of its actions, which it can use to make decisions about which tool to use and what parameters to pass to it.

The agent's memory is what allows it to store the results of its actions, which it can use to make decisions about which tool to use and what parameters to pass to it.

The agent's memory is what allows it to store the results of its actions, which it can use to make decisions about which tool to use and what parameters to pass to it.

The agent's memory is what allows it to store the results of its actions, which it can use to make decisions about which tool to use and what parameters to pass to it.

The agent's memory is what allows it to store the results of its actions, which it can use to make decisions about which tool to use and what parameters to pass to it.

The agent's memory is what allows it to store the results of its actions, which it can use to make decisions about which tool to use and what parameters to pass to it.

The agent's memory is what allows it to store the results of its actions, which it can use to make decisions about which tool to use and what parameters to pass to it.

The agent's memory is what allows it to store the results of its actions, which it can use to make decisions about which tool to use and what parameters to pass to it.

The agent's memory is what allows it to store the results of its actions, which it can use to make decisions about which tool to use and what parameters to pass to it.

The agent's memory is what allows it to store the results of its actions, which it can use to make decisions about which tool to use and what parameters to pass to it.